

AI Doubt Solver & Study Helper

Complete Technical Documentation

Version 1.0 | 2025

RAG Chatbot	Quiz Generator	IMP Questions
Question Predictor	Offline Fallback	PDF Processing

Powered by Google Gemini AI • FAISS • Sentence Transformers • Streamlit

Built with Python • Student-Friendly Design

Table of Contents

1.	Project Overview	
2.	Key Features	
3.	Tech Stack	
4.	How It Works — System Flow	
5.	Prompt Engineering	
6.	User Interface Description	
7.	How to Run the Project	
8.	Sample Input / Output	
9.	Future Improvements	

1 Project Overview

The **AI Doubt Solver & Study Helper** is an intelligent, PDF-powered learning assistant built for students. It lets users upload any academic PDF — textbook, notes, or syllabus — and immediately ask questions, generate quizzes, extract important topics, and predict exam questions, all without needing to read the document manually.

Problem It Solves

- Students waste hours reading large PDFs
- Hard to identify what is important for exams
- No instant doubt-clearing mechanism
- Revision is slow and unstructured

Who Can Use It

- College & university students
- Competitive exam aspirants
- Teachers preparing study material
- Self-learners and researchers

2 Key Features

1 RAG Chatbot — PDF Q&A

- Upload any PDF and ask questions in plain English
- Uses Retrieval-Augmented Generation for accurate answers
- Finds the most relevant chunks from your document
- Powered by Google Gemini AI for natural responses
- Maintains context across multiple questions

2 Quiz Generator

- Auto-generates MCQs from your PDF content
- Choose topic, difficulty, and number of questions
- Instant scoring and answer explanations
- Great for self-testing before exams

3 Important Insights (IMP Questions)

- Identifies high-priority questions likely to appear in exams
- Extracts key concepts, definitions, and formulas
- Presented in a clean, scannable format
- Based on frequency and relevance analysis

4

Predict Question Paper

- Generates a mock exam paper from the PDF
- Includes mix of short, long, and analytical questions
- Mimics real exam patterns and weightage
- Downloadable format for practice

5

Fallback System — Offline Working

- Works even when Google Gemini API is unavailable
- Local processing using FAISS + keyword matching
- Structured answers with headings and bullet points
- No quality compromise — students always get a response

3

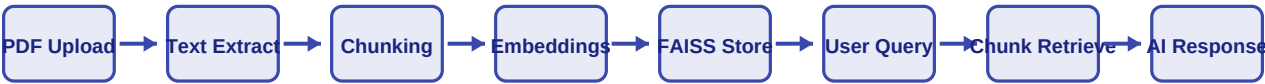
Technology Stack

Technology	Role	Why It Was Chosen
Python 3.10+	Core language	Rich ML/NLP ecosystem, easy to prototype
Streamlit	Web UI framework	Rapid app development, no frontend code needed
FAISS	Vector search engine	Ultra-fast similarity search on embeddings
Google Gemini AI	LLM for responses	State-of-the-art language understanding & generation
PyMuPDF (fitz)	PDF text extraction	Fast, accurate extraction including complex layouts
Sentence Transformers	Embedding model	Creates semantic vector representations of text chunks

4

How It Works — System Flow

4.1 Primary Flow (Online — Gemini API Active)



- PDF Upload:** User uploads a PDF file via the Streamlit interface.
- Text Extraction:** PyMuPDF (fitz) reads every page and extracts raw text while preserving structure.
- Chunking:** Text is split into overlapping chunks (e.g., 500 tokens, 50-token overlap) to preserve context.
- Embeddings:** Each chunk is passed through Sentence Transformers to generate a 768-dim semantic vector.

FAISS Storage: All vectors are indexed in a FAISS flat-L2 index stored in memory for instant retrieval.

User Query: Student types a question in the chat interface.

Chunk Retrieval: The query is embedded and top-k most similar chunks are retrieved from FAISS.

AI Response: Relevant chunks + the query are sent to Gemini. The model returns a well-structured answer.

4.2 Fallback Flow (Offline — API Unavailable)



FALLBACK SYSTEM — Special Feature

Input: User question + PDF extracted text + top relevant chunks (FAISS / keyword match)

Trigger: Gemini API returns error or times out

Step 1: Top-k chunks selected by keyword frequency overlap

Step 2: Noise removal — stop words, repeated phrases cleaned

Step 3: Important sentences ranked by TF-IDF-like scoring

Output: Structured response: Heading | Key Points | Explanation | Summary

Result: Student always receives a readable, useful answer — no blank screen

5 Prompt Engineering

RAG Chat Prompt

You are an expert study assistant. Answer the student's question using ONLY the context below. Be clear, concise, and structured.

Context:
{relevant_chunks}

Student Question: {user_question}

Answer with: Heading, Key Points, Explanation, Summary.

Quiz Generator Prompt

Generate {num_questions} multiple-choice questions from the context.

Topic: {topic} Difficulty: {difficulty}

Context:
{context}

Format each question as:

Q:

A) ... B) ... C) ... D) ...

Correct:

Explanation:

Important Questions Prompt

Identify the {n} most important exam questions from this content.
Focus on definitions, concepts, and frequently tested areas.

Context:
{context}

Format: Numbered list. Mark HIGH PRIORITY items with [*].

Question Paper Prediction Prompt

Generate a {marks}-mark exam paper from the given content.
Include: 5 short answers (2m), 3 medium answers (5m), 2 long answers (10m).

Context:
{context}

Format as a proper exam paper with sections and instructions.

6 User Interface Description

Dashboard

- Landing screen shown after PDF upload
- Displays: file name, page count, word count, detected topics
- Navigation sidebar with buttons for each feature
- Upload progress bar and success confirmation message

Chat Interface

- Conversational layout — user messages on right, AI on left
- Persistent chat history within the session
- Source highlight: shows which PDF section was used
- Copy / regenerate buttons on each response

Quiz Section

- Topic selector and difficulty slider at the top
- MCQ cards rendered one at a time or all at once
- Click to reveal answer with colour-coded feedback (green / red)
- Score summary card shown at the end of the quiz

Important Insights Section

- Ranked list of high-priority questions and concepts
- Star / bookmark each item for personal revision list
- Collapsible explanation for each insight
- Export to text or copy-to-clipboard button

Predict Question Paper Section

- Select marks total, subject area, and exam type
- Generated paper displayed in a clean exam format

- Downloadable as plain text or PDF
- Regenerate button for a different paper variant

7

How to Run the Project

Step 1 — Clone the Repository

```
git clone https://github.com/your-username/ai-doubt-solver.git
cd ai-doubt-solver
```

Step 2 — Create Virtual Environment (Recommended)

```
python -m venv venv
source venv/bin/activate      # Linux / Mac
venv\Scripts\activate        # Windows
```

Step 3 — Install Dependencies

```
pip install -r requirements.txt
```

Step 4 — Add Your API Key

```
# Create a .env file in the project root:
GOOGLE_API_KEY=your_gemini_api_key_here
```

Step 5 — Launch the App

```
streamlit run app.py
```

requirements.txt — Core Dependencies

```
streamlit
google-generativeai
faiss-cpu
sentence-transformers
PyMuPDF
python-dotenv
numpy
```

8

Sample Input / Output

8.1 Chat Q&A Example

User Question	What is gradient descent and how does it work?
Retrieved Chunks	Section 3.2 — Optimization Algorithms (similarity: 0.91) Section 3.4 — Loss Function Minimization (similarity: 0.87)

AI Answer

Gradient Descent — Optimization Algorithm

- Core idea: iteratively move parameters in the direction that reduces loss
- Formula: $\theta = \theta - \text{learning_rate} * \text{gradient}(\text{Loss})$
- Types: Batch GD, Stochastic GD (SGD), Mini-batch GD

Summary: GD finds the minimum of a function by following the slope downhill.

8.2 Quiz Example

Q1. What is the primary purpose of FAISS?

A) Natural language generation

B) Image classification

C) Efficient similarity search

D) Database querying

Correct: C

Explanation: FAISS (Facebook AI Similarity Search) is optimised for fast nearest-neighbour lookups on high-dimensional vectors.

Q2. Which layer in a neural network produces the final output?

A) Input layer

B) Hidden layer

C) Output layer

D) Dropout layer

Correct: C

8.3 Sample Predicted Question Paper

UNIT TEST – Artificial Intelligence & Machine Learning

Total Marks: 30 Time: 1 Hour

SECTION A – Short Answer (2 marks each)

1. Define overfitting and state two ways to prevent it.

2. What is a confusion matrix?

3. Differentiate supervised and unsupervised learning.

SECTION B – Medium Answer (5 marks each)

4. Explain the backpropagation algorithm with a diagram.

5. Compare LSTM and GRU architectures.

SECTION C – Long Answer (10 marks)

6. Describe the end-to-end pipeline of a RAG system.

9 Future Improvements

Better UI / UX • Dark mode support with theme switcher • Mobile-responsive layout for phone access • Drag-and-drop PDF upload with preview	Faster Processing • Async chunking and embedding pipeline • GPU-accelerated FAISS index for large documents • Pre-caching embeddings on first upload
---	---

More Accurate Answers • Fine-tune embedding model on academic corpus • Re-ranking with cross-encoder for better chunk selection • Multi-hop reasoning for complex multi-step questions	Multi-User System • User accounts with saved PDFs and chat history • Shared study rooms for collaborative Q&A; • Teacher dashboard to track student activity
Additional Features • Support for DOCX, PPT, and image-based PDFs (OCR) • Multilingual support — Hindi, regional languages • Voice input / output for accessibility • Spaced repetition reminders for quiz revision	